

Security & Data Model

Encryption, API Keys, Sandboxing and Permissions

• Encryption, API Keys, Sandboxing and Permissions

Category: Security & Data Architecture **Priority:** ★★★★★

[NOTE] This paper describes the security model and data structures of WhyCremisi. It complements Paper 05 (Protocol) and Paper 02 (Architecture). Technologies mentioned are subject to change based on final deployment requirements.

• 1. Security Overview

WhyCremisi operates in a hybrid environment: local processes (plugin bridge, DAW), local network (OSC), and potential cloud connections (AI provider). Each surface exposes specific attack vectors.



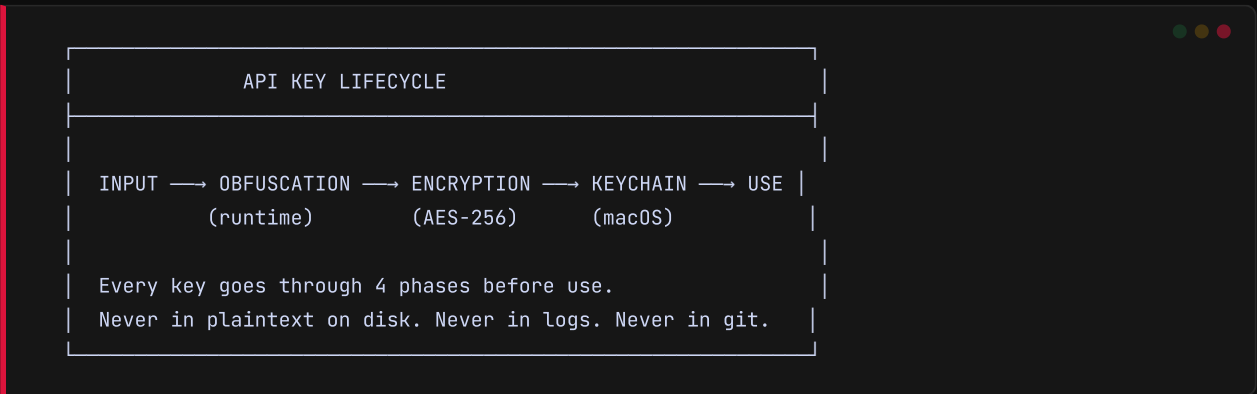
Approach: Defense-in-Depth. No single mechanism is sufficient. Security is layered across 5 levels:

LEVEL 5 – Audit & Monitoring	(detection)
LEVEL 4 – Application	(sandboxing, permissions)
LEVEL 3 – Communication	(TLS, WSS, encryption)
LEVEL 2 – Storage	(AES-256-GCM, keychain)
LEVEL 1 – Physical/OS	(system, user access)

VECTOR	IMPACT	MITIGATION
Key Leak	Critical	Keychain + env vars + rotation
Network MITM	High	TLS 1.3 + WSS
Malicious plugin	High	Sandboxing + timeout + rollback
Disk access	Medium	AES encryption at rest
Session hijack	Medium	Per-session token + expiry
Log exposure	Low	Automatic secret redaction

• 2. API Key Management

API keys are the gateway to AI providers (Gemini, OpenAI, Anthropic). Exposure means loss of control over the agent.



— 2.1 KEYCHAIN INTEGRATION (MACOS)

API keys are stored in **macOS Keychain** (Service `whycremisi-keys`), not in configuration files.

```
// APIKeyManager.cpp – pseudocode
class APIKeyManager {
    SecKeychainRef keychain;

    void storeKey(const std::string& provider, const std::string& key) {
        // SecKeychainAddGenericPassword with process-only ACL
        // Attributes: provider="gemini|ollama|openai|anthropic"
        // Accessible only by the WhyCremisi process
    }

    std::string retrieveKey(const std::string& provider) {
        // 1. Try keychain
        // 2. Fallback: env var WHYCREMISI_KEY_<PROVIDER>
        // 3. Fallback: encrypted config in ~/.whycremisi/config.enc
        // NEVER: log the key
    }

    void rotateKey(const std::string& provider, const std::string& newKey) {
        // Hot rotation without restart
        // Old key retained for 60s for in-flight requests
    }
};
```

— 2.2 ENCRYPTION AT REST

For systems without keychain (Linux, Windows), keys are encrypted in a configuration file:

```
~/.whycremisi/
├─ config.enc           ← AES-256-GCM encrypted
├─ config.enc.salt      ← PBKDF2 salt (16 bytes)
└─ config.enc.nonce     ← AES-GCM nonce (12 bytes)

Key derivation: PBKDF2-HMAC-SHA256, 600,000 iterations
Cipher:         AES-256-GCM (authenticated, 16-byte tag)
Unlock:         Key derived from user password + salt
```

— 2.3 IN-MEMORY OBFUSCATION

At runtime, the key must not remain in plaintext memory longer than necessary:

```
// SecureString – RAII wrapper with zeroisation
class SecureString {
    std::unique_ptr<char[]> data;
    size_t length;

    ~SecureString() {
        // volatile write to prevent compiler optimisation
        volatile char* p = data.get();
        for (size_t i = 0; i < length; i++) p[i] = 0;
    }
};
```

— 2.4 KEY ROTATION

- ▶ Manual rotation: via UI (Settings → API Keys)
- ▶ Automatic rotation: on suspicious usage detection
- ▶ Grace period: 60s overlap between old and new key
- ▶ Notification: UI alert if a key is nearing expiry

— 2.5 NON-STORAGE RULES

WHERE NOT TO STORE	WHY
<code>.env</code> files	Easy accidental git commit
Log files	Visible in logging consoles
Git history	Permanent, not rewritable
Crash dumps	Analysed by third parties
DAW preferences	Accessible by other plugins
AI session	Sent to provider as prompt

Golden rule: if data leaves the WhyCremisi process, it must not contain plaintext API keys.

• 3. Data Encryption

— 3.1 ENCRYPTION AT REST (PERSISTENT STORAGE)

All sensitive persistent data uses **AES-256-GCM**:

```
{
  "algorithm": "AES-256-GCM",
  "key_size": 256,
  "nonce_size": 12, // bytes, NIST recommended
  "tag_size": 16, // GCM authentication tag
  "derivation": "PBKDF2-HMAC-SHA256 x 600000 iterations",
  "mode": "AEAD (encryption + authentication)"
}
```

Encrypted data:

- ▶ API keys (config.enc file)
- ▶ User profiles (user_profile.json.enc) — [NOTE] opt-in
- ▶ Session tokens
- ▶ Plugin discovery cache

Unencrypted data (performance-critical):

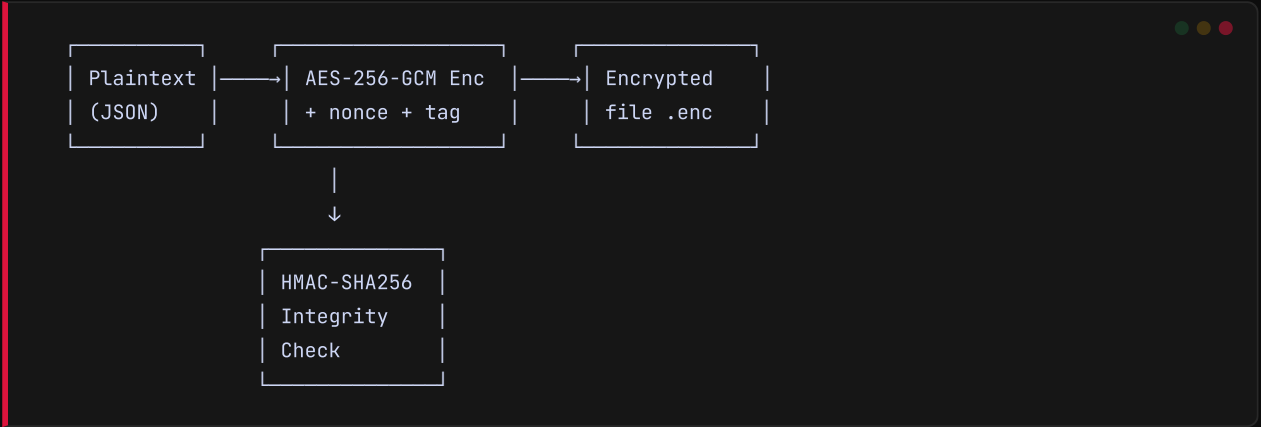
- ▶ Current session state (JSONL)
- ▶ Audit logs (see §8)
- ▶ UI preferences

— 3.2 NETWORK ENCRYPTION

PROTOCOL	CIPHER	PORT	AUTHENTICATION
WebSocket	WSS (TLS 1.3)	443	Certificate + token
OSC (network)	TLS 1.3 μ TLS	8910	Mutual certificate
OSC (local)	None	8910	Localhost only

OSC on localhost can forego encryption since traffic never leaves the machine. For networked OSC (multi-machine), **µTLS** with self-signed certificates exchanged at pairing time is used.

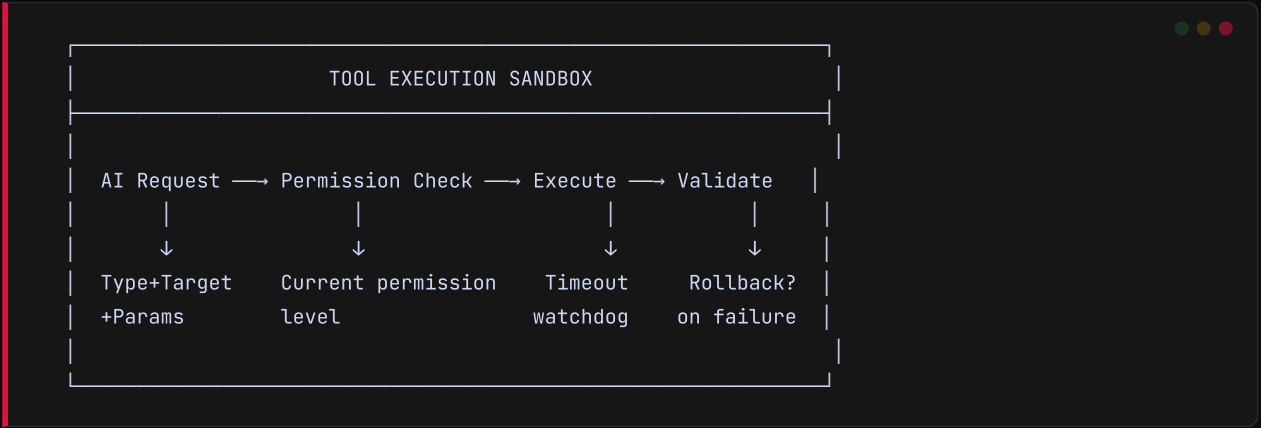
— 3.3 FILE ENCRYPTION SCHEME



• 4. Execution Sandboxing

WhyCremisi executes tool calls (DAW commands, parameter changes, script execution) that can have irreversible effects on the musical project. Every tool call is sandboxed.

— 4.1 EXECUTION ARCHITECTURE



— 4.2 PER-TOOL TIMEOUT

Every tool call has a hard timeout:

TOOL TYPE	DEFAULT TIMEOUT	MAX TIMEOUT
<code>daw.command</code> (play/stop)	2s	5s
<code>daw.command</code> (tempo/volume)	1s	3s
<code>plugin.control</code>	500ms	2s
<code>plugin.query</code>	300ms	1s
<code>ai.prompt</code>	15s	60s
User script	5s	30s

- 1. Tool is forcefully terminated
- 2. State is restored to previous (where possible)
- 3. Error is logged in audit
- 4. User is notified

— 4.3 ROLLBACK ON FAILURE

ROLLBACK MECHANISM

1. State snapshot → capture PRE-modification parameters

2. Tool execution

3. If success → confirm, update snapshot

4. If failure → restore PRE parameters

5. If rollback fails → EMERGENCY state, user alert

Snapshot: { trackId, pluginId, paramName, oldValue, timestamp, contextHash }

— 4.4 PERMISSION LEVELS (TOOL EXECUTION)

LEVEL	DESCRIPTION	DEFAULT	REQUIRES
read-only	Query only, no modification	✓	—
suggest	Show suggestion, manual execution	—	Confirmation
auto-execute	Auto-execute if confidence > 85%	—	Config
full-control	Execute any command without confirm	—	Config + warning

[NOTE] `full-control` is intended for expert users only. Its activation requires explicit confirmation with a risk warning. Audit logs include a special `full_control:true` flag.

• 5. Permission Model

— 5.1 PERMISSION MATRIX

The matrix defines for each user which actions are allowed on which targets:

	read-only	suggest	auto-exec	full-control
Play/Stop	✓	✓	✓	✓
Transport	✓	✓	✓	✓
Track Volume	✓	✓	✓	✓
Mute/Solo	✓	✓	✓	✓
EQ Param	✓*	✓	opt-in	✓
Comp Param	✓*	✓	opt-in	✓
FX Insert	✓*	—	—	✓
FX Remove	✓*	—	—	✓
Script Exec	—	—	opt-in	✓
Master Limiter	✓*	✓	—	✓

* = read-only on current values

— 5.2 GRANULARITY

Permissions are configurable at 3 granularity levels:

LEVEL 1 – Global (per action)
e.g. "EQ Param" → suggest for ALL tracks

LEVEL 2 – Per Track/Plugin
e.g. "EQ Param → auto-execute on Track 1 (Kick)"
"EQ Param → read-only on Track 3 (Vocals)"

LEVEL 3 – Per Specific Parameter
e.g. "Master Limiter.Gain → read-only"
"Master Limiter.TruePeak → suggest"

— 5.3 PER-SESSION OVERRIDE

The user can grant temporary overrides for the current session:

```
{
  "sessionOverrides": [
    {
      "target": "track:3:plugin:Pro-Q3:param:Band1_Gain",
      "level": "auto-execute",
      "reason": "fast workflow while producing",
      "expiresAt": 1716503600000
    }
  ]
}
```

— 5.4 ACTION AUDIT LOG

All executed actions (including automatic ones) are recorded:

```
{
  "timestamp":      1716500000000,
  "userId":         "whyed",
  "action":         "plugin.control",
  "target":         "track:1:plugin:Pro-Q3:param:Band1_Gain",
  "oldValue":       0.0,
  "newValue":       -4.5,
  "permissionLevel": "auto-execute",
  "confidence":     0.92,
  "status":         "success",
  "sessionId":      "abc-123"
}
```

• 6. Data Model

Complete data structures of the WhyCremisi system.

— 6.1 PLUGIN RECORD

FIELD	TYPE	DESCRIPTION
<code>id</code>	UUID v4	Unique identifier
<code>name</code>	string	Plugin name (e.g. "Pro-Q3")
<code>vendor</code>	string	Manufacturer (e.g. "FabFilter")
<code>version</code>	string	Plugin version
<code>paramCount</code>	int	Number of exposed parameters
<code>category</code>	enum	eq, compressor, reverb, limiter, synth, utility, other
<code>tags</code>	string[]	Free tags ("vintage", "analog")
<code>knownProcedures</code>	Procedure[]	Known procedures (see Paper 03)

```
{
  "id": "a1b2c3d4-...",
  "name": "Pro-Q3",
  "vendor": "FabFilter",
  "version": "3.2.1",
  "paramCount": 48,
  "category": "eq",
  "tags": ["digital", "transparent", "surgical", "linear-phase"],
  "knownProcedures": []
}
```


— 6.2 SESSION RECORD

FIELD	TYPE	DESCRIPTION
<code>id</code>	UUID v4	Unique session ID
<code>daw</code>	string	DAW name ("REAPER", "Logic Pro", etc.)
<code>tracks</code>	Track[]	Project tracks array
<code>plugins</code>	Plugin[]	Plugins loaded in session
<code>aiState</code>	object	Agent state at session start
<code>createdAt</code>	timestamp	Creation time
<code>lastActivity</code>	timestamp	Last recorded action

```
{
  "id": "session-001",
  "daw": "REAPER",
  "tracks": [
    {
      "id": 1,
      "name": "Kick",
      "color": "#FF4400",
      "pluginSlots": [
        { "slot": 0, "pluginId": "a1b2c3d4-...", "active": true },
        { "slot": 1, "pluginId": "e5f6g7h8-...", "active": true }
      ]
    }
  ],
  "plugins": [],
  "aiState": {
    "personality": "professional_warm",
    "memoryVersion": 2
  },
  "createdAt": 1716500000000,
  "lastActivity": 1716503600000
}
```

— 6.3 PRESET RECORD

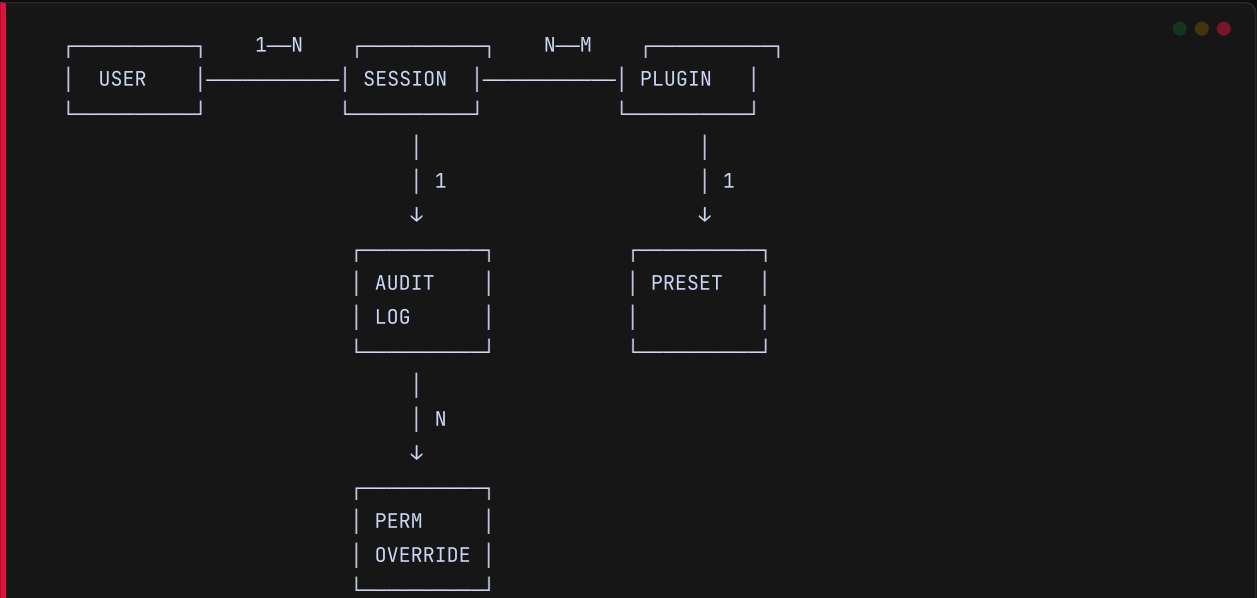
FIELD	TYPE	DESCRIPTION
<code>id</code>	UUID v4	Unique identifier
<code>name</code>	string	Preset name
<code>pluginId</code>	UUID v4	Reference to Plugin
<code>params</code>	map<string, float>	Parameter name → value map
<code>tags</code>	string[]	Search tags
<code>createdAt</code>	timestamp	Creation date
<code>usageCount</code>	int	Times used

```
{
  "id": "preset-001",
  "name": "Kick Tight 808",
  "pluginId": "a1b2c3d4-...",
  "pluginName": "Pro-Q3",
  "params": {
    "Band1_Freq": 55.0,
    "Band1_Gain": 4.0,
    "Band1_Q": 1.2,
    "Band2_Freq": 220.0,
    "Band2_Gain": -3.5,
    "Band2_Q": 8.0
  },
  "tags": ["kick", "tight", "808", "sub-bass"],
  "createdAt": 1716400000000,
  "usageCount": 12
}
```

6.4 AUDITENTRY RECORD

FIELD	TYPE	DESCRIPTION
timestamp	timestamp	When the action occurred
userId	string	User identifier
action	string	Action type (e.g. plugin.control)
target	string	Specific target
oldValue	any?	Previous value (if applicable)
newValue	any?	New value (if applicable)
status	enum	success, error, warning, timeout, rollback
details	string	Additional notes
permissionLevel	string	Permission level used

6.5 ENTITY RELATIONSHIPS



• 7. Privacy

WhyCremisi is designed to be **local-first**. Audio data never leaves the user's machine.

— 7.1 CORE PRINCIPLES

PRINCIPLE	IMPLICATION
Local-first	AI can run fully offline (Ollama). Audio data: never in cloud.
Analytics opt-in	Telemetry disabled by default. User must explicitly enable it.
Transparency	User can see exactly what data is being collected at all times.
Right to deletion	User can delete ALL their data at any time.

— 7.2 WHAT IS NEVER COLLECTED

- Raw audio samples
- Project names (unless analytics opt-in)
- User file metadata
- Content of non-WhyCremisi plugins
- Keystrokes or UI interactions unrelated to WhyCremisi

— 7.3 RETENTION POLICY

DATA	RETENTION	DELETION
Audit log	90 days	<code>/audit/clear</code> or automatic
User profile	While active	<code>/reset</code>
Session JSONL	30 days	Automatic rotation
API keys	While valid	Manual rotation
Analytics	12 months	Immediate opt-out
Plugin cache	7 days	Recreated on demand

— 7.4 RIGHT TO DELETION

<code>/reset</code>	→ reset profile, sessions, permissions
<code>/forget <pattern></code>	→ delete specific memory
<code>/audit/export</code>	→ export audit log as JSON
<code>/audit/clear</code>	→ clear audit log
<code>/data/export</code>	→ export all user data
<code>/data/delete</code>	→ delete all data (GDPR-style)

• 8. Audit Logging

Every action executed by the system is tracked in an immutable append-only audit log.

— 8.1 AUDIT ENTRY STRUCTURE

```
{
  "timestamp":      1716500000000,
  "type":           "plugin.control",
  "userId":         "whyed",
  "sessionId":      "session-001",
  "target":         "track:1:plugin:Pro-Q3:param:Band1_Gain",
  "before":         0.0,
  "after":          -4.5,
  "status":         "success",
  "permission":     "auto-execute",
  "confidence":     0.92,
  "duration_ms":    120,
  "ip":             "127.0.0.1",
  "userAgent":      "WhyCremisi/2.0",
  "details":        "EQ cut suggested by agent, auto-accepted"
}
```

— 8.2 LOG ROTATION

```
~/whycremisi/audit/
├─ audit.log           ← Current (append)
├─ audit.2025-05-12.json ← Rotated daily
├─ audit.2025-05-11.json
├─ audit.2025-05-10.json.gz ← Compressed after 7 days
└─ ...

> Rotation:    Daily, at midnight
> Compression: gzip after 7 days
> Retention:   90 days (auto-delete afterwards)
> Max size:    100MB per file (immediate split if exceeded)
```

— 8.3 EXPORT AND UI

COMMAND	FORMAT
/audit/export	JSON array
/audit/export?format=csv	CSV
/audit/export?since=7d	Last 7 days only
/audit/export?type=error	Errors only
/audit/search?q=Pro-Q3	Text search

UI: Audit Panel in Settings with:

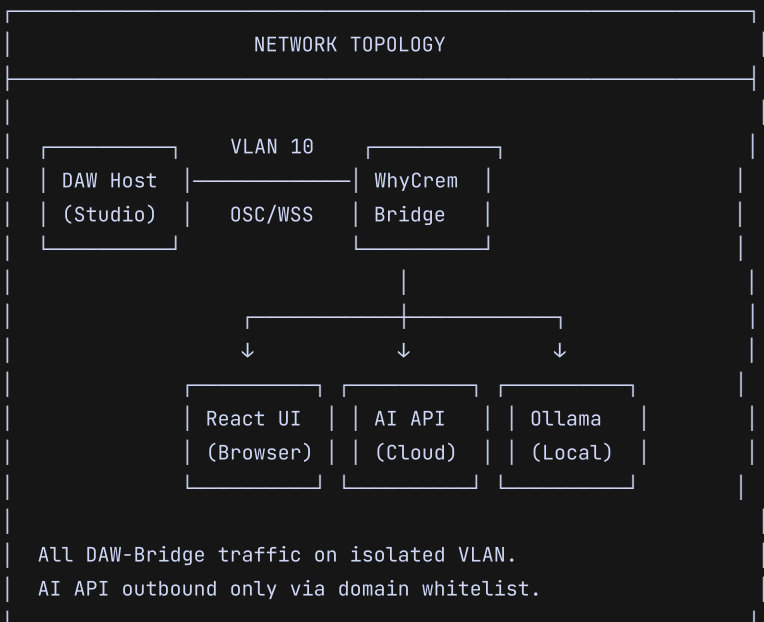
- Interactive timeline
- Filters by type/status/date
- Expandable detail rows
- Export button

• 9. Deployment Recommendations

— 9.1 PRODUCTION HARDENING CHECKLIST

- ❑ TLS 1.3 configured on all remote connections
- ❑ Signed certificates (Let's Encrypt or internal CA)
- ❑ pTLS for multi-machine communication
- ❑ API keys in vault / 1Password CLI / Doppler
- ❑ Log rotation and retention configured
- ❑ Rate limiting on APIs (100 req/min per provider)
- ❑ Tool call timeouts configured for environment
- ❑ User permissions set (default: suggest)
- ❑ Sandboxing active and verified
- ❑ Monitoring endpoints /health and /metrics
- ❑ Alerting on: error rate > 5%, timeout > 10%, key rotation needed
- ❑ Encrypted backup of config.enc

— 9.2 NETWORK SEGMENTATION



— 9.3 SECRETS MANAGEMENT

TOOL	USE	BENEFIT
macOS Keychain	Local API keys	Natively secure
1Password CLI	CI/CD secrets	Audit + rotation
HashiCorp Vault	Multi-machine	Dynamic secrets
Mozilla SOPS	Encrypted GitOps	Git-friendly encryption
Doppler	Team deployment	Environment inheritance

```
KEY METRICS (prometheus format)
-----
whycremisi_tool_calls_total{status="success|error|timeout"}
whycremisi_api_key_last_rotation{provider="gemini"}
whycremisi_sandbox_timeout_total
whycremisi_audit_entries_total
whycremisi_websocket_connections_active
whycremisi_permission_blocks_total

ALERT THRESHOLDS
-----
● Critical: error rate > 10% over 5 min
● Warning:  avg response time > 2x baseline
● Info:      key rotation needed (> 80% validity)
```

— 9.5 SECURITY REVIEW CADENCE

```
► Automated: Every PR runs SAST (Semgrep) + dependency audit
► Manual:    Quarterly threat model review
► Pentest:   Annual network & API surface testing
► Bounty:    Invitation-only bug bounty program
```

• 10. Summary

WhyCremisi security rests on three pillars: **prevention** (encryption, sandboxing, permissions), **detection** (audit logging, monitoring), and **response** (rollback, rotation, alerting). The threat model is designed for a professional music production environment where the potential damage is not a data breach but the corruption of an artistic project.

SECURITY LAYERING SUMMARY		
PREVENTION	DETECTION	RESPONSE
AES-256-GCM	Audit log	Rollback
TLS 1.3 / WSS	Monitoring	Key rotation
Sandbox + timeout	Alerting	User notify
Permission matrix	/audit/export	/reset
Keychain + env var	Search log	
Memory obfuscation		
Defense-in-depth		

[NOTE] Security is a process, not a state. This paper will be updated with every threat model revision. Current version is v1.0, valid for the alpha and beta phases of the project.

